Feasibility study on the interactive AI scheduling with LLM technology for human-centric Industry 5.0

Hyun-jin Oh[1[0009-0003-2836-4877]], Ji-won Baek[1[0009-0006-6920-0192]], Ki-young Cho[1[0009-0009-6556-2067]] and Jong-hun Woo[1,2[0000-0002-7612-7361]]

[1] Department of Naval Architecture and Ocean Engineering, Seoul National University, Seoul 08826, Republic of Korea
[2] Research Institute of Marine Systems Engineering, Seoul National University, Seoul 08826, Republic of Korea

Abstract. As the manufacturing industry rapidly shifts toward high-mix, large-scale production and extreme personalization, scheduling optimization has become significantly more complex than before. Existing search-based methods such as mathematical optimization, metaheuristics, and constraint programming face fundamental limitations in handling the high variability and uncertainty that characterize real-world manufacturing environments. Deep neural network technologies are emerging at the forefront. When coupled with the human-centric manufacturing paradigm of Industry 5.0, there is growing anticipation that Large Language Models (LLMs), capable of interpreting constraints and objectives expressed in natural language, could lead to more flexible and sustainable manufacturing ecosystems. In this study, we propose a method for solving manufacturing system scheduling optimization by utilizing an LLM, a form of unsupervised deep neural networks. Specifically, we focus on the Job Shop Scheduling Problem (JSSP), known to be an NP-hard task, and introduce a process wherein the JSSP is represented in natural language and then trained with an LLM using Low-Rank Adaptation (LoRA) and an enhanced prompting technique. Our results show that this approach achieves scheduling performance comparable to existing neural network methods, thereby suggesting the potential for LLM-based scheduling optimization to serve as a key tool in human-centric manufacturing under Industry 5.0.

Keywords: Large Language Model, Industry 5.0, Job Shop Scheduling Problem,

1 Introduction

1.1 Background & Significance

The modern manufacturing system faces the challenge of potential unsustainability. Mass production has contributed to environmental pollution and climate change, while traditional numerical KPI-prioritizing approaches have exacerbated income gaps and social inequality. These emerging sustainability concerns have highlighted Industry 5.0, emphasizing environmental sustainability, human wellbeing, and resilience.

Industry 5.0 introduced the term "Human-centric Smart Manufacturing (HSM)" as an alternative manufacturing approach. Moreover, with Large Language Model (LLM) advent, HSM conceptual discussions have expanded into technical issues, as LLM communicates most akin to human interaction, satisfying human-centric Industry 5.0 philosophy.

LLM replaces human cognitive tasks like logical reasoning by using vast data surpassing human capacity. An emerging LLM use case in manufacturing is scheduling. Traditionally, scheduling, a combinatorial optimization problem (COP), involves sequencing numerous tasks and allocating resources to the right task. Scheduling is critical for resource utilization optimization and, more broadly, for minimizing costs and maximizing profits. However, solution space size grows exponentially with task count. Moreover, checking solution feasibility under spatiotemporal constraints exceeds human computational limits, making scheduling highly time-consuming.

To overcome these challenges, various techniques (e.g., heuristics, metaheuristics, search techniques) have been employed. However, mathematical formulations of real-world problems these approaches exploit often fail to capture implicit human worker needs and preferences.

The Industry 5.0 philosophy represents a shift from a technology-oriented, numerically measurable KPI-prioritizing approach to a human-centric system integrating human experience and tacit knowledge. In this context, we argue LLM can achieve this philosophy via computational capability, accessibility, and agility. LLMs are built on a computational structure enabling vast data processing, strong reasoning, generalization capabilities, and can thus generate optimal COP solutions. Their natural language processing ability allows complex communication with manufacturing workers, even without specialized optimization expertise, while effectively incorporating real-world objectives and constraints. Additionally, as LLMs are linked to interactive networks, the entire process can be online. This connectivity enables LLMs to interact with digital twins, dynamically adapting to real-time changes and updated schedules.

This paper aims to demonstrate LLM's capability as an effective scheduler for manufacturing systems under the Industry 5.0 philosophy. Specifically, the Job Shop Scheduling Problem (JSSP), which is an NP-hard COP that models most manufacturing systems, is tested to assess the effectiveness of LLM. To explore both the capabilities and limitations of LLM as a scheduler for manufacturing systems, we propose a framework that utilizes LLM to generate solutions for JSSP instances. In the proposed framework, the JSSP is effectively reformulated into a natural language prompt to align with an LLM's input format. In addition, various techniques are considered to minimize learning costs and ensure response quality. Experimental results demonstrate that LLM can generate effective, near-optimal schedules when provided with scheduling problems with objectives and constraints in natural language. The results also suggest LLM's potential to act as a human-centric Industry 5.0 scheduler by interacting with workers in manufacturing scenes and incorporating experience-derived objective functions and constraints.

To examine the feasibility of the proposed LLM-based scheduling framework, this study addresses two research questions. The first asks whether an LLM, when prompted with natural-language representations of JSSP, can generate valid schedules satisfying

hard constraints. The second explores how the scheduling performance of an LLM-based approach compares to traditional optimization and learning-based methods, particularly in terms of makespan.

If the proposed approach can generate valid and effective schedules for complex constraint problems such as JSSP, it would suggest the method can also capture implicit knowledge and diverse real-world constraints in practical settings. This implies strong applicability to manufacturing environments such as PCB assembly lines or shipyard block outfitting, where natural-language interaction and iterative schedule adjustments are essential. Accordingly, this study experimentally investigates the feasibility of using an LLM to solve the JSSP.

2 Literature review

2.1 LLM in Manufacturing

LLMs are expected to act as intermediaries between humans and robots in manufacturing, analyzing and organizing information from numerous sources to provide humans necessary insights. They assist with repetitive, simple cognitive tasks traditionally human-performed, ultimately optimizing the manufacturing environment [1,2].

A promising LLM application area is Human-Robot Collaboration (HRC). Characterized as zero-shot learning, LLMs' strong generalization capability enables robots to perform general tasks without specific domain knowledge training, serving as a foundation for human-centric modeling [3]. In manufacturing systems, LLMs can facilitate seamless interaction when robots automate human tasks in bottleneck areas. For example, LLMs can be used to automate AGVs to tackle repetitive tool retrieval problems that cause disruptions between assembly operations in manufacturing shops [4].

Another potential application is enhancing Digital Twin integration. As LLMs connect to networks allowing real-time access to vast information, they can interactively retrieve and summarize only necessary human data. This enables LLMs to effectively bridge the gap in using digital twins for human decision-making [5]. LLMs can also aid manufacturing scheduling tasks. Scheduling, an optimization sector involving mathematical reasoning, requires verifying if LLMs can generate logically consistent, correct inferences. A study showed LLMs could perform assembly line task planning given natural language instructions [6].

Furthermore, when combined with industrial low-code software, LLMs exhibited the ability to generate processing plans for completing tasks in manufacturing environments. In the proposed framework, LLMAPM (Large Language Model-based Adaptive Process Management), an approach for generating structured process plans via task decomposition and step-wise synthesis using fine-tuned LLMs, LLMs successfully integrated various machinery and generated task descriptions in the required format. However, human intervention was still required for manual coding [7]. LLMs could extend planning scope to the machinery level, successfully extracting requirements from natural language prompts, deciding parameters, and designing machine tool paths [8]. However, in manufacturing chatbots, LLMs struggled with long, consecutive mathematical reasoning tasks [1].

To overcome this, numerous studies proposed integrated frameworks combining LLMs with other techniques for enhanced reasoning and response consistency. Wu et al. incorporated LLMs with the ACTR cognitive architecture, effectively implementing human decision-making. Their study found LLMs capable of understanding vector-represented latent information from the cognitive architecture for optimization-related decisions [9]. Zhao et al. developed a multiple LLM agent framework, each with a specific decision-making role. This allowed each agent to focus on specific tasks within complex manufacturing problems needing various decision steps like allocation and sequencing. Tested on a Flexible Job Shop Problem dataset, the framework showed superior performance over heuristic methods [10].

Many studies applying LLMs to manufacturing see them as tools replacing tedious, physical, and repetitive human tasks. Even for cognitive tasks, LLM performance expectations are often limited to human logical reasoning levels. However, LLM potential extends beyond this, possibly solving optimization problems surpassing human cognitive and reasoning capabilities—like combinatorial optimization problems requiring exploration of an exponentially increasing solution space, or decision-making tasks involving vast data statistical analysis. Whether LLMs hold mathematical optimizer potential is further examined next.

2.2 Large Language Models for Combinatorial Optimization

Recent studies actively explored solving challenging combinatorial optimization problems using LLMs. Various combinatorial optimization problem types exist—like the Traveling Salesman Problem (TSP), Vehicle Routing Problem (VRP), and Job Shop Scheduling Problem (JSSP)—most known as NP-hard.

For TSP and routing, Yang et al. (2023) proposed Optimization by Prompting (OPRO), using an LLM as an iterative optimization tool [11]. Liu et al. (2024) introduced LMEA, integrating an LLM into the genetic algorithm loop; prompts facilitate parent selection, crossover, and mutation for new TSP routes [12]. Another Liu et al. (2024) work proposed AEL, combining an LLM and evolutionary operations to evolve the Guided Local Search (GLS) heuristic for TSP [13]. Iklassov et al. (2024) introduced Self-Guiding Prompting, enabling LLM to generate multiple solution paths simultaneously, then backtrack and modify [14].

In Vehicle Routing Problems (VRP), Huang et al. (2024) showed LLM feasibility for robotic routing [15]. Kikuta et al. (2024) proposed Route Explainer, a retrospective VRP solution explanation framework [16], and Huang et al. (2024) presented a multimodal textual-visual approach [17].

For JSSP and Flow Shop Scheduling, Amarasinghe et al. (2023) introduced AI-Copilot. They used finetuned GPT-3 to divide problems into token-limited modules, generating a MILP model per module [18]. Mu et al. (2025) proposed a hybrid LLM-Monte Carlo Tree Search (MCTS) approach for Flow Shop Scheduling, generating initial heuristics and iteratively refining performance [19]. Abgaryan et al. (2024) presented a JSSP natural-language dataset [20].

In Bin Packing and Knapsack problems, Ye et al. (2024) proposed Language Hyper-Heuristics (LHHs) and a Reflective Evolution (ReEvo) LLM framework [21]. Romera-

Paredes et al. (2024) developed Fun Search, integrating an LLM with a systematic evaluator, discovering new knowledge on difficult problems [22].

For graph optimization like coloring, Wang et al. (2023) introduced NLGraph, a benchmark for evaluating natural language graph problem-solving [23]. Huang et al. (2025) proposed GraphThought, finetuning a smaller LLM on a Thought dataset [24].

These studies show LLM potential as combinatorial optimization tools. We propose a JSSP problem-solving approach using LLMs by generating natural language problem statements, employing LoRA training, and incorporating self-improvement.

3 Problem definition

In this study, we select the JSSP as it can comprehensively capture various constraints and dynamic factors in real world manufacturing settings, and propose a novel problem representation that restructures it in natural language.

3.1 Job Shop Scheduling Problem (JSSP)

The JSSP involves multiple jobs that must be processed on different machines in a predetermined sequence, and its most basic form is defined as follows.

Given n jobs, m machines ($n, m \geq 1$), each job j consists of a series of operations $\{O_{j,1}, O_{j,2}, \ldots\}$. Each operation can be performed on only one specific machine, has a fixed processing time, and follows a predefined order for each job.

The optimization objective of this problem is to minimize the total time required to complete all jobs, commonly referred to as the makespan. Details regarding the notation used in this paper are provided in Table 1.

As one of the most representative and widely applied scheduling problems in real-world manufacturing environments, the JSSP has been a key issue in improving productivity and optimizing resources across diverse industries such as assembly operations, automotive production, and shipbuilding. Because it can comprehensively account for complex job sequences, machine assignments, and interdependencies among jobs, the JSSP effectively reflects real-world manufacturing challenges. For this reason, this study adopts the JSSP as the target problem to explore the feasibility of using an LLM as a scheduler.

3.2 Natural Language Represented Job Shop Scheduling Problem

Traditional JSSPs are generally described using matrices or numerical data. In this study, we present an approach that restructures both JSSP instances and their constraints into a natural language format, aiming to effectively capture real-world scheduling complexity in a way that is more amenable to large language model (LLM) learning [25]. We use CPlex, an optimization solver provided by IBM, to generate optimal solutions for JSSP sizes ranging from 2-job/2-machine to 10-job/10-machine problems and then convert these solutions into natural language. This process yields a dataset that includes job sequences, machine assignments, processing times, and constraints,

enabling LLMs to learn from a wide range of scheduling scenarios. An example is provided in Table 2.

Table 1. Indices and Notations for JSSP

Indices	Explanations
n	Number of jobs
m	Number of machines
j	Job index ($1 \leq j \leq n$)
i	Machine index ($1 \leq i \leq m$)
Notations	**Explanations**
$O_{j,k}$	k −th operation of job
$p_{j,k}$	Processing time of the k −th operation of job
$M_{j,k}$	Machine assigned to the k −th operation of job
$S_{j,k}$	Start time of the k −th operation of job
$C_{j,k}$	Completion time of the k −th operation of job
C_{max}	The maximum completion time among all operations

Table 2. Natural-Language Represented Job Shop Scheduling Problem

Natural-Language Represented Job Shop Scheduling Problem Example
Given a JSSP with 8 jobs and 3 machines, the goal is to minimize the makespan In the format Operation X: MY, Z: X is the operation number (starting from 0). Y is the machine number on which this operation must be performed. Z is the processing time required for this operation. Job 0 consists of Operations: Operation 0: M1,53 Operation 1: M3,67 … Solution: Job 0 Operation 0, M1 → $0 + 53 = 53$ Job 1 Operation 0, M3 → $0 + 48 = 48$ …

4 Methodology

This study proposes an integrated framework employing an LLM, trained on a natural-language reformulation of JSSP, as an optimization solver for manufacturing system scheduling. Traditional scheduling methods require explicit mathematical formulation of problem constraints, making it difficult to incorporate tacit knowledge or enable flexible, human-centric reasoning. Unlike conventional approaches—such as mathematical optimization, dispatching rules, or reinforcement learning—our method addresses this research gap by directly interpreting natural language scheduling

problems and generating solutions via prompt-driven sequence modeling. The overall architecture is illustrated in Fig. 1, where the LLM operates in training and inference: fine-tuned with natural-language–solution pairs using LoRA, it subsequently generates, evaluates, and iteratively refines scheduling sequences. The framework incorporates both hard constraints and tacit operational knowledge, enabling adaptive improvement through self-reflective prompting. It comprises four main components: model selection, prompt template design, learning technique, and self-reflective prompting.

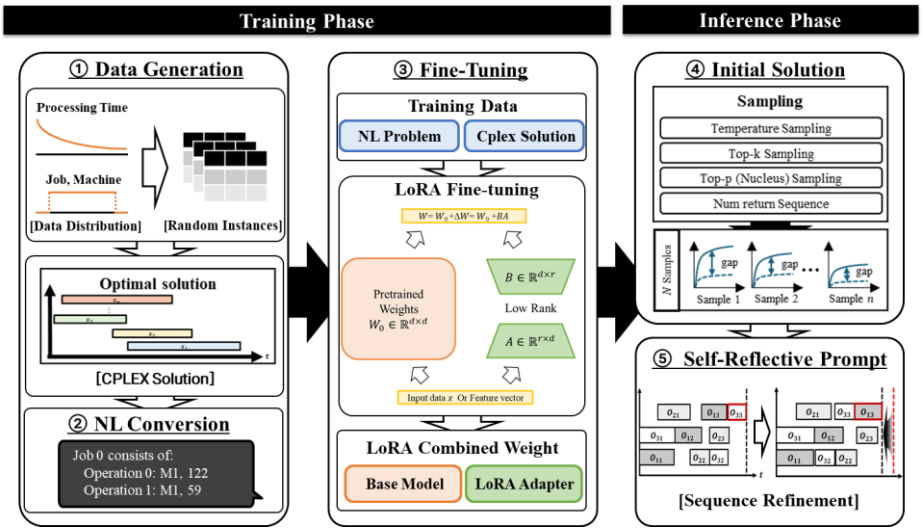

Fig. 1. End-to-end architecture of the proposed LLM-based scheduling framework

4.1 Model Selection

In this study, we adopt the LG Exaone 3.5 7.8B model to effectively use the proposed learning framework. Several factors motivated this choice:

Its approximately 7.8 billion parameters significantly reduce computational resources compared to larger language models while maintaining excellent performance; notably, instruction tuning and preference alignment maximize user command processing, and its Korean and English optimization offers high compatibility with domestic manufacturing environments. Additionally, a model with high token capacity is essential for the natural language problem descriptions used in this study, including job sequences and processing times; this model supports up to 32k tokens, enabling efficient handling of complex problem data and ensuring detailed work conditions and constraints are fully captured. Furthermore, the model has demonstrated outstanding performance in mathematical and logical benchmarking tasks, indicating strong capabilities in mathematical problem-solving and logical reasoning tasks, and this strength suits it for deriving high-quality solutions to manufacturing scheduling problems.

For these reasons, LG Exaone 3.5 7.8B was selected for its lightweight architecture, long context window, and robust mathematical reasoning abilities, collectively making

it an optimal choice for addressing complex optimization challenges in real-world manufacturing environments.

4.2 Prompt Template for Learning

The training prompt is organized into four stages: three are for guiding the model to understand and solve the Job Shop Scheduling Problem effectively, and one shapes the output into the desired format.

a) Natural Language Represented Job Shop Scheduling Problem

This stage provides a JSSP description in natural language, derived from solutions generated by the CPlex mathematical optimization tool. Each problem includes details such as Jobs, Operations, Machines, and Processing times, helping the model intuitively grasp the problem's structure.

b) System Message: Rules, Format, Strategy

This message consists of constraints, an explanation of the operation format, and a task sequencing strategy

- Constraints: A list of mandatory rules for solving the JSSP.
- Operation Format Explanation: A detailed guide, with examples, on how each operation should be specified.
- Selection Strategy: Guidance on how to arrange tasks to minimize the makespan.

These three elements are merged into a single system role message that clearly communicates the key constraints, formats, and strategies needed to solve the problem.

c) User Message (Prompt Variants & Problem)

One of several "Prompt Variants" is randomly selected and combined with the natural language problem description. By varying the sentence structures in these prompts, the model learns to respond to different ways of asking the same question.

d) Assistant Message (Expected Output)

Finally, an example answer is inserted as the assistant message, enabling the model to recognize both the output format and solution criteria. The example includes the sequence of tasks for each machine and the makespan, thereby directly guiding the model's generation objectives during training.

By presenting the system message, the user message, and the assistant message in a structured order, we establish a cohesive chat template. This format enables the model to learn both the problem's rules and real input-output examples simultaneously. Such a prompt template is crucial for helping the model acquire the necessary domain

knowledge for problem-solving and produce accurate scheduling solutions for similar cases in the future. The overall structure is shown in Fig. 2.

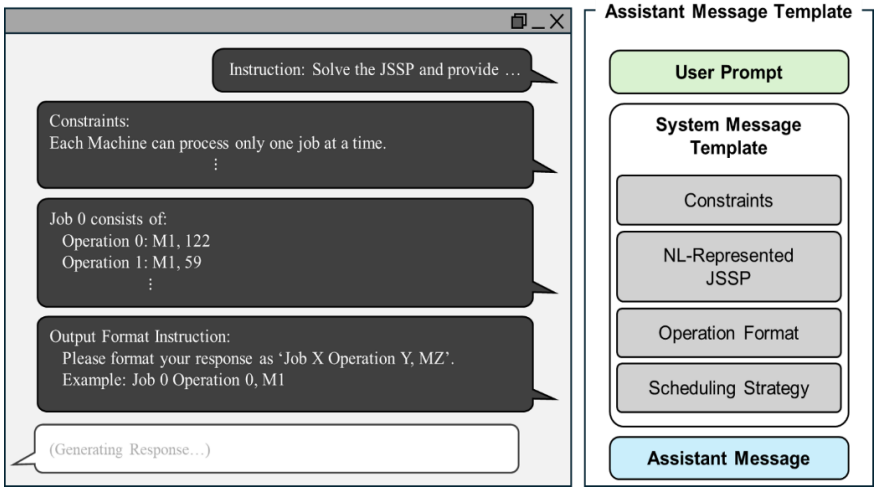

Fig. 2. Prompt Template for Learning structure

4.3 Learning Technique

a) LoRA Fine-Tuning (Low-Rank Adaptation)

Conventional fine-tuning, which updates all parameters of a LLM, demands substantial computational resources and memory. To address this issue, this study adopts LoRA Fine-Tuning, one of the parameter-efficient fine-tuning (PEFT) techniques. LoRA Fine-Tuning keeps the base weights W_0 of a pretrained model fixed and represents the weight update ΔW using two learnable low-rank matrices A and B. The final model weights are defined as:

$$W = W_0 + \Delta W = W_0 + BA \tag{1}$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ are learnable low-dimensional matrices, and r (the rank) is a small integer satisfying $\mathrm{r} \ll \min(d, k)$.

Suppose W_0 is a $(d \times d)$. matrix. Setting a small integer rank r makes B a $(d \times r)$ matrix and A a $(r \times d)$ matrix, so BA matrix becomes $(d \times d)$ in size, matching W_0. Because only $2dr$ additional parameters $(d \times r + r \times d)$ are need to be learned instead of all d^2 parameters of W_0, the required computation is significantly reduced. Thus, LoRA Fine-Tuning can effectively capture problem-specific information without updating every parameter, leading to large savings in computational load and memory consumption. Fig. 3 illustrates both the Full Fine-Tuning and LoRA Fine-Tuning architectures.

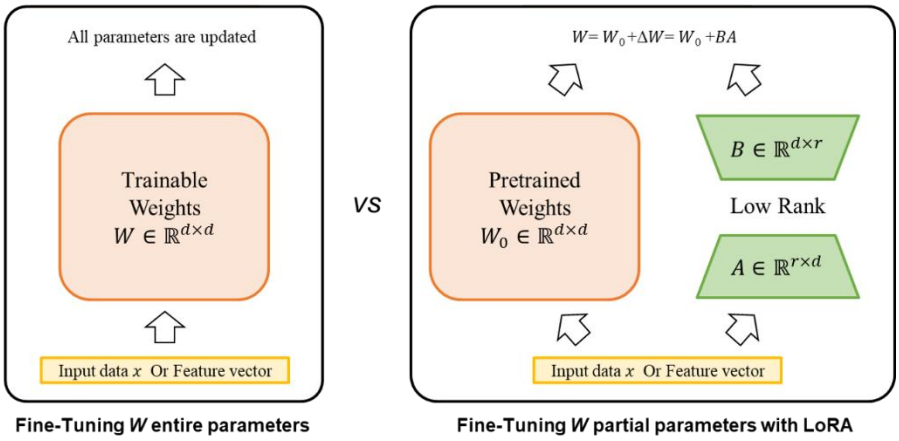

Fig. 3. Full Fine-Tuning, LoRA Fine-Tuning Structure

b) Model Compression Strategy

Solving manufacturing scheduling problems with large language models requires substantial computational resources and memory. In GPU-constrained environments, especially when real-time problem-solving is necessary, it becomes vital to reduce model size to ensure both fast inference and efficient memory use. This study applies two primary compression techniques: 4-bit quantization and gradient checkpointing.

4-bit quantization effectively decreases GPU memory consumption during training by representing parameters with lower precision. Compared to 32-bit precision, this method can reduce memory usage by up to eight times, and compared to 16-bit precision, by up to four times. Meanwhile, gradient checkpointing avoids storing all intermediate activations during the training process and recalculates them only when needed, significantly reducing memory requirements.

By applying these compression strategies, we can train more efficiently in resource-limited GPU environments while maintaining strong performance.

4.4 Self-Reflective Prompt

To guide the LLM toward better scheduling results based on an initially generated solution, we designed a self-reflective prompt. This prompt feeds the existing solution back into the LLM and encourages it to propose an alternative task sequence that reduces the makespan. By having the model analyze its own previous output and build upon it, we foster the generation of an optimized task sequence. Notably, we include the condition “do not change the machine assignments,” allowing the schedule to be adjusted while respecting machine constraints.

In our experiments, the self-reflection stage is repeated only once. The overall prompt format is shown in Table 3.

Table 3. Self-Reflective Prompt for Schedule Optimization Prompt Example

Self-Reflective Prompt for Schedule Optimization Prompt Example
This is an initial solution to the current JSSP. Analyze the result, identify potential inefficiencies, and reschedule the operations to reduce the overall makespan. **Insert the Current Natural-Language JSSP Problem** **Insert the initial solution generated by the LLM** **Instruction:** Re-evaluate the above schedule and propose an improved operation sequence. Your revised solution should aim to minimize the makespan.

5 Training and Evaluation

5.1 Training

Using the methods described above, training was performed in the Vessl AI MLOps environment on a single GPU (NVIDIA A100–40GB) with a micro-batch size of 1 and 64 gradient accumulation steps. The LoRA fine-tuning parameters rank r and α were both set to 128. We set the learning rate to 2e-4, epoch to 2 and given the GPU specifications, used 22,500 samples for training and 2,250 for validation. These values were selected through empirical tuning to balance convergence stability and computational feasibility under limited GPU resources, while ensuring optimal performance under the given constraints. Fig. 4 shows the training and validation curves, and the gradient plots.

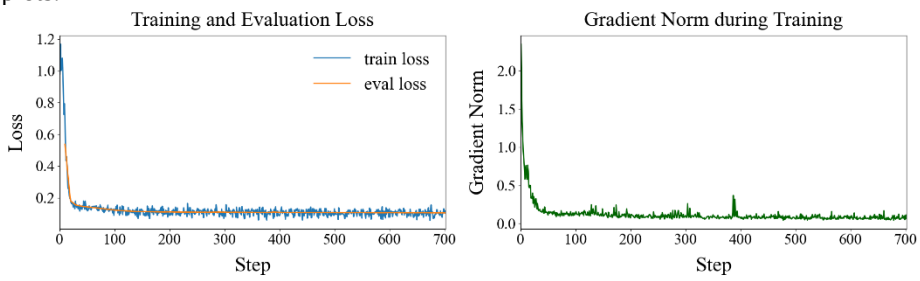

Fig. 4. Train/eval loss, gradient norm graph

5.2 Evaluation

During the evaluation stage, we parse and verify model-generated JSSP solutions. Using regular expressions, we extract from the output string each job's ID, operation number, machine number, start time, processing time, end time, and the makespan.

Subsequently, we perform a single improvement round and then proceed to constraint-checking.

To assess performance, we compare our method against heuristic and reinforcement-learning approaches. The heuristic methods used are well-known priority-based rules widely applied in the JSSP; for instance, SPT selects the job requiring the shortest processing time first; LPT prioritizes the job with the longest processing time first; MWKR determines priority based on each job's total remaining operations, processing the job with the largest amount of unfinished work first; and FIFO chooses the operation that became ready first among all currently eligible operations.

Additionally, we conducted a comparative experiment using L2D [26], a reinforcement-learning-based method. Test problems are from Taillard (TA) [27] and Lawrance (LA) [28]. We measure performance on each benchmark using the Percentage Relative Difference (PRD), calculated per Equation (2).

$$PRD = \frac{100 \times (C - C^m)}{C^m} \tag{2}$$

Where C is the makespan obtained by the algorithm, and C^m is the optimal makespan for that instance. Tables 4 and 5 show a comparison of the LLM model's initial output and self-improved output against SPT, LPT, MWKR, FIFO, and L2D.

Table 4. Evaluation of Different Algorithms Using the Taillard(TA) Benchmark Set

	SPT	MWKR	FIFO	L2D	Initial Solution	Improved Solution
TA01	52.1	45.1	23.3	18.0	18.2	17.2
TA02	37.8	55.8	20.3	22.5	20.1	18.1
TA03	65.2	60.3	22.8	18.6	22.0	22.0
TA04	40.3	44.8	30.0	38.8	19.5	19.4
TA05	67.1	54.2	26.2	32.2	20.1	20.1
TA06	43.1	59.2	18.0	29.2	24.5	19.8
TA07	74.5	60.4	17.1	27.2	24.2	21.5
TA08	36.1	58.1	29.2	21.1	18.3	18.3
TA09	54.9	73.6	24.5	28.5	23.5	21.1
TA10	75.5	56.5	21.6	23.4	24.5	19.5
AVG	**54.6**	**56.8**	**23.3**	**26.0**	**21.5**	**20.0**
TA11	62.5	56.0	31.7	32.7	26.1	19.8
TA12	76.5	62.8	44.8	31.8	25.4	21.2
TA13	75.1	51.4	11.5	44.1	19.1	19.1
TA14	57.3	61.5	28.2	23.2	26.5	21.5
TA15	61.4	62.3	26.4	28.5	21.1	20.3
TA16	63.1	86.4	24.1	26.4	23.1	19.5
TA17	49.1	38.8	29.3	29.5	25.8	18.8
TA18	62.9	64.2	28.5	28.5	19.4	19.4
TA19	68.5	65.3	23.0	26.3	25.3	19.8
TA20	76.1	62.9	25.1	29.1	21.4	17.7
AVG	**65.2**	**61.2**	**27.2**	**30.0**	**23.3**	**19.7**

Table 5. Evaluation of Different Algorithms Using the Lawrance(LA) Benchmark Set

	SPT	LPT	FIFO	Initial Solution	Improved Solution
LA01	29.0	33.5	15.2	14.3	12.5
LA02	35.6	36.5	27.3	22.5	21.7
LA03	29.0	25.3	25.1	20.1	20.1
LA04	37.5	43.7	20.0	19.8	19.5
LA05	39.4	32.7	0.0	10.1	9.5
AVG	**34.1**	**34.3**	**17.5**	**17.4**	**16.7**
LA06	47.8	19.3	0.0	5.1	4.2
LA07	26.7	21.2	23.1	17.3	17.3
LA08	35.3	22.9	13.6	14.2	11.5
LA09	35.5	16.8	0.0	2.5	2.5
LA10	37.0	18.6	0.0	0.0	0.0
AVG	**36.5**	**19.8**	**7.3**	**7.8**	**7.1**

From the experimental results, it is evident that our method outperformed both heuristic and reinforcement learning approaches. Moreover, when the improvement prompt was applied, the resulting performance improved compared to the initial solution. This outcome also suggests that additional rounds of self-reflection could unlock further gains, illustrating the broader potential of LLM-based scheduling approaches.

6 Conclusion

We propose a learning framework for utilizing a large language model (LLM) to solve the Job Shop Scheduling Problem (JSSP) described in natural language. This framework demonstrates that LLMs can play a key role in human-centric manufacturing systems in three ways. First, we show that an LLM can accept optimization problems, whose constraints and objectives are expressed in natural language, and generate appropriate solutions in the required format. Second, by combining fine-tuning with an open-source LLM architecture, the model can fully utilize its inherent generalization ability while effectively solving combinatorial optimization problems. Finally, by minimizing the training overhead, the model's parameters can be updated at any time, enabling continuous adaptation to dynamic manufacturing environments and newly introduced constraints.

Our experiments with the LG Exaone 3.5 7.8B model show that LoRA fine-tuning enables the LLM to solve natural-language JSSP instances using minimal computational resources. Moreover, the solutions generated are of high quality: the proposed LLM-based approach surpasses traditional heuristic methods in various scales of JSSP, including the Taillard and Lawrance benchmarks, and remains competitive with L2D, a reinforcement-learning-based method. We also confirm that applying an improvement prompt can further enhance solution quality. By successfully tackling the complex JSSP, our results suggest that the proposed LLM can also accept constraints and objective functions in natural language for other manufacturing environments, producing effective schedules.

Finally, we plan to extend our approach to other problems such as the Permutation Flow Shop Problem (PFSP) and Flexible Job Shop Scheduling Problem (FJSP), using a natural language dataset and LoRA training. This study used solutions generated by a CP optimizer as training data. However, we acknowledge that as the problem size increases, computational cost becomes significant and the optimizer's search strategy may introduce inherent biases. To address this and better handle large-scale and dynamic scheduling problems, future work will avoid relying solely on optimization-generated data and instead explore self-labeling, GRPO-based methods, or reinforcement learning techniques that do not require labeled solutions—thus enhancing diversity in the solution space and reducing systematic bias. Additional investigations will include techniques to enhance data collection and reasoning processes, security methods, explainable AI (XAI), and interactive prompting strategies to mathematically formulate tacit knowledge in collaboration with human operators—bringing us closer to achieving truly human-centric manufacturing systems in the Industry 5.0 era

Acknowledgments. This research was supported by (a) the Ministry of Trade, Industry and Energy of the Republic of Korea (Project No. P0022929), (b) the National Research Foundation of Korea (NRF) grant funded by the Korean government (Grant No. RS-2025-00555741), and (c) the Research Institute of Marine Systems Engineering at Seoul National University.

Disclosure of Interests. The authors have no conflicts of interest to declare.

References

1. Garcia, C.I., DiBattista, M.A., Letelier, T.A., Halloran, H.D., Camelio, J.A.: Framework for LLM applications in manufacturing. Manufacturing Letters 41, 253–263 (2024)
2. Shi, F., Zhang, Y., Qu, C., Fan, C., Chu, J., Jin, L., Liu, S.: Leveraging the power of large language models to drive progress in the manufacturing industry. In: 9th International Conference on Financial Innovation and Economic Development (ICFIED 2024). pp. 125–133. Atlantis Press (2024)
3. Li, S., Wang, Z., Yan, Z., Gao, Y., Jiang, H., Zheng, P.: Large Language Model for Humanoid Cognition in Proactive Human-Robot Collaboration. In: 2024 IEEE 20th International Conference on Automation Science and Engineering (CASE). pp. 540–545. IEEE (2024)
4. Wang, T., Fan, J., Zheng, P.: An LLM-based vision and language cobot navigation approach for Human-centric Smart Manufacturing. Journal of Manufacturing Systems 75, 299–305 (2024)
5. Šturm, J., Zajec, P., Škrjanc, M., Mladenić, D., Grobelnik, M.: Enhancing Cognitive Digital Twin Interaction using an LLM Agent. In: 2024 47th MIPRO ICT and Electronics Convention (MIPRO). pp. 103–107. IEEE (2024)
6. Gkournelos, C., Konstantinou, C., Makris, S.: An LLM-based approach for enabling seamless Human-Robot collaboration in assembly. CIRP Annals 73(1), 9–12 (2024)
7. Ni, M., Wang, T., Leng, J., Chen, C., Cheng, L.: A large language model-based manufacturing process planning approach under industry 5.0. International Journal of Production Research, 1–20 (2025)
8. Fan, H., Liu, X., Fuh, J.Y.H., Lu, W.F., Li, B.: Embodied intelligence in manufacturing: leveraging large language models for autonomous industrial robotics. Journal of Intelligent Manufacturing, 1–17 (2024)

9. Wu, S., Oltramari, A., Francis, J., Giles, C.L., Ritter, F.E.: Cognitive LLMs: Towards Integrating Cognitive Architectures and Large Language Models for Manufacturing Decision-making. arXiv preprint arXiv:2408.09176 (2024)
10. Zhao, Z., Tang, D., Zhu, H., Zhang, Z., Chen, K., Liu, C., Ji, Y.: A Large Language Model-based multi-agent manufacturing system for intelligent shopfloor. arXiv preprint arXiv:2405.16887 (2024)
11. Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q.V., Zhou, D., Chen, X.: Large language models as optimizers. arXiv:2309.03409 (2023)
12. Liu, S., Chen, C., Qu, X., Tang, K., Ong, Y.-S.: Large language models as evolutionary optimizers. In: 2024 IEEE Congress on Evolutionary Computation (CEC) (2024)
13. Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., Zhang, Q.: An example of evolutionary computation large language model beating human: Design of efficient guided local search. CoRR (2024)
14. Iklassov, Z., Du, Y., Akimov, F., Takac, M.: Self-guiding exploration for combinatorial problems. Advances in Neural Information Processing Systems 37, 130569–130601 (2024)
15. Huang, Z., Shi, G., Sukhatme, G.S.: Can Large Language Models Solve Robot Routing? arXiv:2403.10795 (2024)
16. Kikuta, D., Ikeuchi, H., Tajiri, K., Nakano, Y.: RouteExplainer: An Explanation Framework for Vehicle Routing Problem. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining (2024)
17. Huang, Y., Zhang, W., Feng, L., Wu, X., Tan, K.C.: How multimodal integration boost the performance of llm for optimization: Case study on capacitated vehicle routing problems. arXiv:2403.01757 (2024)
18. Amarasinghe, P.T., Nguyen, S., Sun, Y., Alahakoon, D.: AI-copilot for business optimisation: A framework and a case study in production scheduling. arXiv:2309.13218 (2023)
19. Mu, C., Zhang, X., Wang, H.: Planning of Heuristics: Strategic Planning on Large Language Models with Monte Carlo Tree Search for Automating Heuristic Optimization. arXiv:2502.11422 (2025)
20. Abgaryan, H., Harutyunyan, A., Cazenave, T.: Llms can schedule. arXiv:2408.06993 (2024)
21. Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., Song, G.: Reevo: Large language models as hyper-heuristics with reflective evolution. arXiv:2402.01145 (2024)
22. Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J., Ellenberg, J.S., Wang, P., Fawzi, O.: Mathematical discoveries from program search with large language models. Nature 625(7995), 468–475 (2024)
23. Wang, H., Feng, S., He, T., Tan, Z., Han, X., Tsvetkov, Y.: Can language models solve graph problems in natural language? Advances in Neural Information Processing Systems 36, 30840–30861 (2023)
24. Huang, Z., Guo, L., Sheng, J., Chen, H., Li, W., Jin, B., Lu, C., Wang, X.: GraphThought: Graph Combinatorial Optimization with Thought Generation. arXiv:2502.11607 (2025)
25. Li, J., Guo, D., Yang, D., Xu, R., Wu, Y., He, J.: CodeI/O: Condensing Reasoning Patterns via Code Input-Output Prediction. arXiv:2502.07316 (2025)
26. Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C.: Learning to dispatch for job shop scheduling via deep reinforcement learning. In: 34th Conference on Neural Information Processing Systems (NeurIPS) (2020)
27. Taillard, E.: Benchmarks for basic scheduling problems. European Journal of Operational Research 64(2), 278–285 (1993)
28. Lawrance, S.: Resource constrained project scheduling: an experimental investigation of heuristic scheduling techniques. GSIA, Carnegie Mellon University (1984)